

Metodologias Ágeis e Scrum

Entregar valor, aprender e adaptar



Prof. Dr. João Choma

Universidade Estadual de Maringá — Engenharia de Software

github.com/JoaoChoma

Aulas 06 e 07



Como equipes organizam trabalho complexo em ciclos curtos de feedback.



Ao final, você deverá ser capaz de:

- explicar por que abordagens ágeis trabalham de forma iterativa e incremental;
- interpretar os valores e princípios do Manifesto Ágil;
- distinguir agilidade de ausência de planejamento;
- reconhecer a estrutura do Scrum;
- relacionar responsabilidades, eventos, artefatos e compromissos;
- acompanhar uma Sprint do objetivo ao Incremento;
- aplicar Scrum a um exemplo prático.



Projetos de software convivem com:

- necessidades que se tornam mais claras durante o uso;
- tecnologia e integrações incertas;
- riscos descobertos durante a implementação;
- prioridades de negócio que mudam;
- aprendizado obtido com entregas reais.

Uma abordagem adaptativa transforma incerteza em ciclos de decisão.



- Abordagem preditiva — Abordagem adaptativa
- busca detalhar o caminho antecipadamente — detalha mais o próximo passo
- controla mudanças em relação ao plano — usa mudanças como fonte de aprendizado
- grandes marcos de entrega — incrementos frequentes
- progresso medido por fases e documentos — progresso observado em resultados utilizáveis
- funciona melhor com alta previsibilidade — favorece trabalho complexo e incerto

Agilidade não elimina planejamento; aproxima planejamento e evidência.



Iterativo e incremental não são sinônimos

- **Iterativo:** revisamos uma solução para melhorá-la com aprendizado.
- **Incremental:** adicionamos partes utilizáveis ao produto.
- **Iterativo e incremental:** entregamos uma parte, coletamos feedback e ajustamos produto e plano.

```
ideia -> incremento -> feedback -> adaptacao
      ^                      |
      +----- novo ciclo -----+
```



Dezessete profissionais reuniram experiências de desenvolvimento de software e formularam quatro valores.

A declaração não afirma que os itens à direita não têm valor. Ela diz que, quando houver tensão, os itens à esquerda devem receber maior valorização.

Fonte oficial: agilemanifesto.org/iso/ptbr/manifesto.html.



- ① **Indivíduos e interações** mais que processos e ferramentas.
- ② **Software em funcionamento** mais que documentação abrangente.
- ③ **Colaboração com o cliente** mais que negociação de contratos.
- ④ **Responder a mudanças** mais que seguir um plano.

Os itens à direita possuem valor. Em uma situação de tensão, entretanto, os itens destacados à esquerda recebem maior valorização.

Pergunta prática: quando processo e resultado entram em conflito, qual escolha aumenta o valor entregue e o aprendizado?



Processos e ferramentas ajudam, mas não substituem:

- responsabilidade compartilhada;
- comunicação clara;
- confiança;
- colaboração entre negócio e tecnologia;
- capacidade de resolver impedimentos.

Exemplo: um quadro atualizado não compensa uma equipe que evita conversar sobre riscos.



Documentos são úteis quando reduzem risco, preservam conhecimento ou apoiam decisões.

A evidência mais forte de progresso, porém, é um resultado que possa ser:

- executado;
- demonstrado;
- validado;
- usado para obter feedback.

Cuidado: código incompleto e não integrado não é um Incremento utilizável.



Colaboração frequente permite:

- validar prioridades;
- esclarecer critérios de aceitação;
- descobrir necessidades;
- testar hipóteses;
- ajustar o produto antes de grandes investimentos.

O contrato define uma relação. A colaboração melhora as decisões dentro dela.



Adaptar não é trocar de prioridade todos os dias.

A equipe precisa de:

- objetivo de produto;
- horizonte de curto prazo estável;
- critérios para ordenar trabalho;
- transparência sobre custo e impacto;
- momentos claros de inspeção e adaptação.

Mudança útil é aquela que incorpora aprendizado e aumenta valor.



Os princípios serão organizados em quatro grupos didáticos. A divisão ajuda a enxergar relações, mas os **12 itens continuam independentes e igualmente importantes**.

- ① **Valor e adaptação:** princípios 1, 2 e 3.
- ② **Colaboração e confiança:** princípios 4, 5 e 6.
- ③ **Progresso e sustentabilidade:** princípios 7, 8 e 9.
- ④ **Simplicidade e melhoria:** princípios 10, 11 e 12.

Como ler: em cada grupo, observe o resultado desejado, o comportamento esperado e a evidência que permite verificar sua aplicação.



- ① **Satisfazer o cliente:** prioridade à entrega antecipada e contínua de software com valor.
- ② **Acolher mudanças:** requisitos podem mudar, mesmo tarde; a mudança deve gerar vantagem para o cliente.
- ③ **Entregar frequentemente:** disponibilizar software funcionando em poucas semanas ou meses, preferindo intervalos menores.

Exemplo prático: entregar primeiro a confirmação de consulta, medir o uso e ajustar o fluxo antes de construir todos os relatórios.

Observe: frequência sem valor apenas produz mais entregas; mudança sem direção apenas cria instabilidade.



- 1 **Trabalhar juntos:** pessoas de negócio e desenvolvimento colaboram diariamente durante o projeto.
- 2 **Apoiar pessoas motivadas:** oferecer ambiente, suporte e confiança para que realizem o trabalho.
- 3 **Conversar diretamente:** a conversa face a face é o meio mais eficiente e eficaz de transmitir informação na equipe.

Exemplo prático: negócio, desenvolvimento e testes refinam juntos uma história e seus critérios de aceitação.

Observe: ferramentas registram decisões, mas não substituem diálogo, contexto e responsabilidade compartilhada.



- 1 **Medir progresso por software funcionando:** resultado utilizável é a principal evidência de avanço.
- 2 **Manter ritmo sustentável:** patrocinadores, usuários e desenvolvedores devem sustentar um ritmo constante indefinidamente.
- 3 **Cuidar da excelência técnica:** atenção contínua à qualidade técnica e ao bom design aumenta a agilidade.

Exemplo prático: integrar e testar a cada entrega, controlando dívida técnica e evitando uma corrida no fim da Sprint.

Observe: velocidade aparente com defeitos, retrabalho ou horas extras recorrentes não representa progresso sustentável.



- 1 **Praticar simplicidade:** maximizar a quantidade de trabalho não realizado é essencial.
- 2 **Favorecer auto-organização:** as melhores arquiteturas, requisitos e designs emergem de equipes auto-organizadas.
- 3 **Refletir e ajustar:** em intervalos regulares, a equipe avalia como ser mais eficaz e adapta seu comportamento.

Exemplo prático: eliminar uma funcionalidade sem uso, deixar a equipe decidir a solução e testar uma melhoria na Sprint seguinte.

Observe: simplicidade não é descuido; auto-organização não é ausência de objetivo; retrospectiva precisa gerar ação verificável.



- 1. entrega de valor contínua;
- 2. mudanças acolhidas com propósito;
- 3. software entregue frequentemente;
- 4. negócio e desenvolvimento juntos;
- 5. pessoas motivadas, apoiadas e confiáveis;
- 6. comunicação direta;

Use o checklist para avaliar uma prática real da equipe. Peça uma evidência para cada item marcado.



- 7. software funcionando mede progresso;
- 8. ritmo sustentável;
- 9. excelência técnica e bom design;
- 10. simplicidade;
- 11. equipes auto-organizadas;
- 12. reflexão e adaptação regulares.



- Scrum;
- Kanban;
- Extreme Programming (XP);
- Lean Software Development;
- Feature-Driven Development;
- Crystal.

Elas podem enfatizar aspectos diferentes: fluxo, feedback, práticas técnicas, organização ou entrega.

Não existe uma combinação universal; o contexto orienta a escolha.



- Abordagem — Ênfase principal
- Scrum — framework para gerar valor em problemas complexos
- Kanban — visualizar e melhorar o fluxo de trabalho
- XP — práticas técnicas e feedback rápido no desenvolvimento
- Lean — reduzir desperdício e otimizar o sistema de entrega

Uma equipe pode usar Scrum e incorporar práticas de XP ou técnicas de visualização de fluxo.



Um MVP não é "qualquer produto incompleto". Ele precisa permitir aprendizado relevante sobre valor, uso ou viabilidade.

Observe: cada etapa deve ser utilizável para o objetivo do experimento.

Imagem extraída da apresentação original; crédito nela indicado: DB1 Global Software.



Vantagens aparecem quando existe feedback real

Para clientes e produto:

- foco no que gera mais valor;
- riscos percebidos mais cedo;
- transparência sobre o estado real;
- possibilidade de ajustar prioridades;
- melhoria progressiva da solução.

Para a equipe:

- autonomia com responsabilidade;
- comunicação frequente;
- problemas visíveis;
- melhoria contínua;
- objetivos de curto prazo claros.



Sinais de alerta:

- usar "ágil" para justificar falta de documentação necessária;
- iniciar tudo e terminar pouco;
- confundir urgência com prioridade;
- medir produtividade por quantidade de tarefas;
- aceitar interrupções constantes;
- realizar reuniões sem inspeção ou adaptação;
- manter baixa qualidade para "entregar mais rápido".

Agilidade exige disciplina, não ausência de processo.



Scrum é um framework leve para ajudar pessoas, equipes e organizações a gerar valor por meio de soluções adaptativas para problemas complexos.

Ele define elementos essenciais e deixa espaço para práticas adequadas ao contexto.

Por isso, testes automatizados, integração contínua, Kanban, refinamento e técnicas de estimativa podem ser usados com Scrum, mas não são definidos pelo Scrum Guide.



Decisões são tomadas com base no que é observado e aprendido.

Os três pilares são:

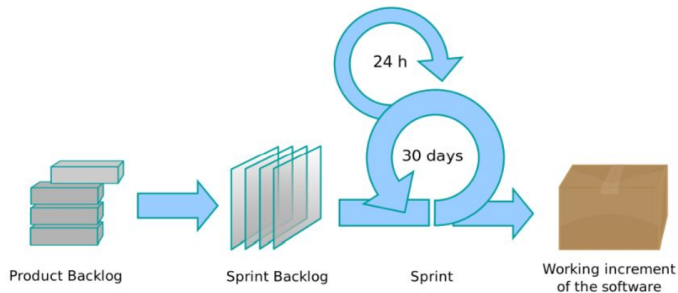
- **transparência:** trabalho e estado precisam ser compreensíveis;
- **inspeção:** produto e processo são observados com frequência;
- **adaptação:** desvios e novos aprendizados mudam o próximo passo.

Sem transparência, a inspeção engana; sem adaptação, inspecionar vira ritual.



- **compromisso** com objetivos;
- **foco** no trabalho da Sprint;
- **abertura** sobre trabalho e desafios;
- **respeito** às capacidades e diferenças;
- **coragem** para enfrentar problemas e tomar decisões difíceis.

Os valores orientam comportamento quando o guia não oferece uma resposta mecânica.



Product Goal

↓ orienta

Product Backlog → Sprint Backlog → Sprint → Incremento

Cada Sprint cria uma oportunidade concreta de verificar direção, produto e modo de trabalho.



A Sprint:

- tem duração fixa de um mês ou menos;
- começa imediatamente após a anterior;
- possui um Sprint Goal;
- contém planejamento, trabalho, Daily Scrum, Review e Retrospective;
- deve produzir ao menos um Incremento utilizável.

Durante a Sprint, mudanças não podem colocar o Sprint Goal em risco.



Maximiza valor e gerencia efetivamente o Product Backlog.

Scrum Master

Estabelece Scrum conforme o guia e apoia a efetividade da equipe.

Developers

Criam o plano da Sprint e produzem um Incremento utilizável.

O Scrum Team é pequeno, multifuncional e autogerenciável; não existem subequipes ou hierarquias internas definidas pelo framework.



O Product Owner é responsável por:

- desenvolver e comunicar o Product Goal;
- criar e comunicar itens do Product Backlog;
- ordenar o backlog;
- garantir transparência e entendimento do backlog.

Trabalho pode ser delegado, mas a responsabilidade permanece com o Product Owner.

Armadilha: transformar o PO em intermediário passivo entre "cliente" e equipe.



O Scrum Master:

- ensina e orienta sobre Scrum;
- ajuda a remover impedimentos;
- apoia autogerenciamento e multifuncionalidade;
- facilita eventos quando necessário;
- trabalha com equipe e organização para melhorar efetividade.

Ele não distribui tarefas diariamente nem controla pessoas pelo quadro.



Developers são responsáveis por:

- criar o Sprint Backlog;
- seguir a Definition of Done;
- adaptar o plano diariamente em direção ao Sprint Goal;
- responsabilizar-se mutuamente como profissionais.

O título não depende do cargo técnico: qualquer pessoa que contribui para criar o Incremento atua nessa responsabilidade.



A equipe responde:

- 1 Por que esta Sprint é valiosa?
- 2 O que pode ser feito nesta Sprint?
- 3 Como o trabalho escolhido será realizado?

O resultado é o Sprint Backlog: Sprint Goal, itens selecionados e plano de entrega.

Imagem extraída da apresentação original.



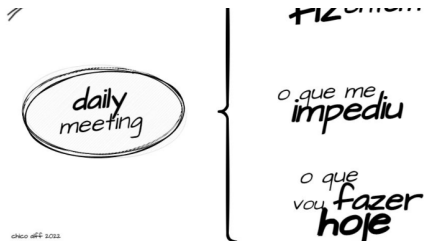
Produto: sistema de clínica universitária.

Sprint Goal: permitir que a recepção encontre horários disponíveis e confirme uma consulta básica.

Possíveis itens:

- pesquisar profissional e data;
- listar horários livres;
- confirmar consulta;
- impedir conflito de horário.

Se surgir uma solicitação de relatório financeiro, ela pode ser importante sem contribuir para o objetivo desta Sprint.



A Daily Scrum:

- ocorre a cada dia útil da Sprint;
- dura 15 minutos;
- é um evento dos Developers;
- inspeciona progresso em direção ao Sprint Goal;
- produz um plano ajustado para o próximo dia.

As três perguntas tradicionais são uma técnica possível, não uma regra obrigatória do Scrum.



Scrum Team e stakeholders:

- observam o resultado da Sprint;
- discutem mudanças no ambiente;
- avaliam progresso em direção ao Product Goal;
- ajustam o Product Backlog.

A Review não é apenas uma apresentação de slides nem uma cerimônia de aceite formal.



A equipe inspeciona:

- pessoas e interações;
- processos e ferramentas;
- qualidade e Definition of Done;
- problemas encontrados e resolvidos;
- mudanças que podem aumentar efetividade.

Uma melhoria importante pode entrar no Sprint Backlog seguinte.



Três artefatos tornam o trabalho transparente

- Artefato — Conteúdo
- Product Backlog — lista emergente e ordenada do que melhora o produto
- Sprint Backlog — objetivo, itens selecionados e plano dos Developers
- Incremento — resultado utilizável que atende à Definition of Done

Cada artefato possui um compromisso que aumenta foco e transparência.



- Artefato — Compromisso
- Product Backlog — Product Goal
- Sprint Backlog — Sprint Goal
- Incremento — Definition of Done
- **Product Goal:** estado futuro desejado do produto.
- **Sprint Goal:** propósito único da Sprint.
- **Definition of Done:** descrição formal da qualidade exigida para o Incremento.

Sem esses compromissos, listas e quadros perdem direção.



Um item só integra o Incremento quando atende à Definition of Done.

Ela pode incluir, por exemplo:

- código revisado;
- testes automatizados aprovados;
- integração realizada;
- critérios de aceitação verificados;
- segurança e acessibilidade mínimas;
- documentação essencial atualizada;
- capacidade de implantação.

Trabalho incompleto retorna ao Product Backlog e não deve ser apresentado como concluído.



Um quadro pode mostrar:

A fazer → Em andamento → Em revisão → Concluído

Ele ajuda a:

- tornar trabalho visível;
- limitar trabalho em andamento;
- revelar bloqueios;
- discutir fluxo.

Scrum não exige uma ferramenta específica. Um quadro ruim pode apenas tornar visível um processo confuso.



- Ordem — Item — Valor ou risco
- 1 — confirmar consulta sem conflito — operação essencial
- 2 — cancelar e liberar horário — evita indisponibilidade incorreta
- 3 — enviar confirmação por e-mail — reduz faltas
- 4 — relatório mensal — necessidade administrativa

Ordenar não é apenas escolher "o mais fácil"; valor, risco, dependência e aprendizado influenciam a decisão.



Métricas úteis respondem perguntas:

- estamos entregando valor?
- quanto tempo o trabalho leva para atravessar o sistema?
- onde há espera ou retrabalho?
- a qualidade está sustentável?
- o Sprint Goal está em risco?

Evite: usar pontos, quantidade de tarefas ou velocidade para comparar pessoas e equipes. Uma métrica vira problema quando passa a ser o objetivo.



- 1 Defina produto, stakeholders e Product Goal.
- 2 Forme um Scrum Team com responsabilidades claras.
- 3 Crie e ordene o Product Backlog.
- 4 Defina uma Definition of Done inicial.
- 5 Escolha uma duração de Sprint.
- 6 Realize Sprint Planning e formule o Sprint Goal.
- 7 Torne trabalho e impedimentos visíveis.
- 8 Inspecione diariamente, entregue um Incremento e obtenha feedback.
- 9 Adapte produto e processo.

Comece simples e preserve os elementos essenciais.



Contexto: clínica universitária precisa reduzir conflitos de agendamento.
Em equipes:

- 1 formule um Product Goal;
- 2 escreva seis itens de Product Backlog;
- 3 ordene os itens e justifique os três primeiros;
- 4 formule um Sprint Goal;
- 5 selecione itens coerentes com o objetivo;
- 6 escreva uma Definition of Done;
- 7 indique o que será inspecionado na Review e na Retrospective.



Checklist para revisar a aplicação do Scrum

- existe Product Goal compreendido?
- Product Backlog está transparente e ordenado?
- Sprint Goal orienta decisões?
- Developers controlam o plano da Sprint?
- Daily Scrum inspeciona progresso, não pessoas?
- Incremento atende à Definition of Done?
- stakeholders participam da Review?
- Retrospective produz adaptação real?
- impedimentos são tratados?
- métricas apoiam aprendizado em vez de punição?



- agilidade trabalha com entrega, feedback e adaptação;
- os valores do Manifesto orientam escolhas, não eliminam os itens à direita;
- Scrum é um framework leve para problemas complexos;
- transparência, inspeção e adaptação sustentam o empirismo;
- Product Owner, Scrum Master e Developers formam um único Scrum Team;
- eventos criam ciclos de feedback;
- artefatos e compromissos conectam trabalho, objetivo e qualidade;
- realizar reuniões não basta: cada inspeção precisa permitir adaptação.



- Manifesto Ágil — agilemanifesto.org/iso/ptbr/manifesto.html
- Princípios do Manifesto — agilemanifesto.org/iso/ptbr/principles.html
- Scrum Guide 2020 — scrumguides.org
- Scrum.org — scrum.org/resources/what-scrum-module